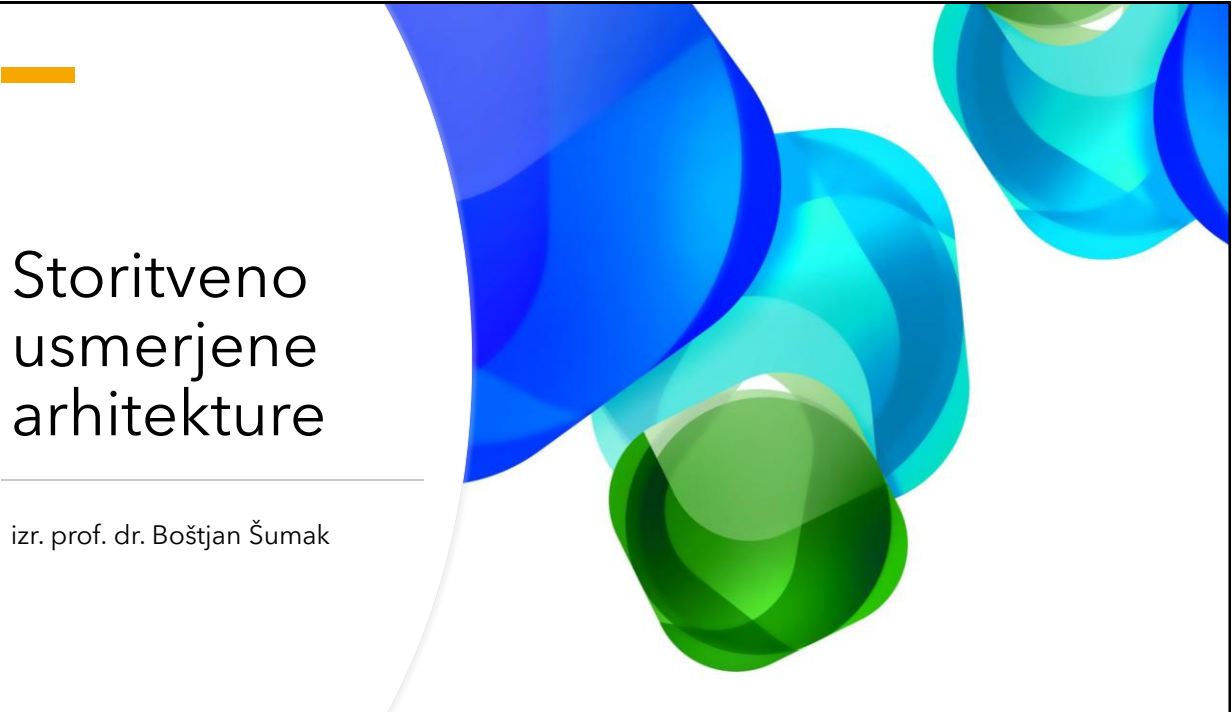
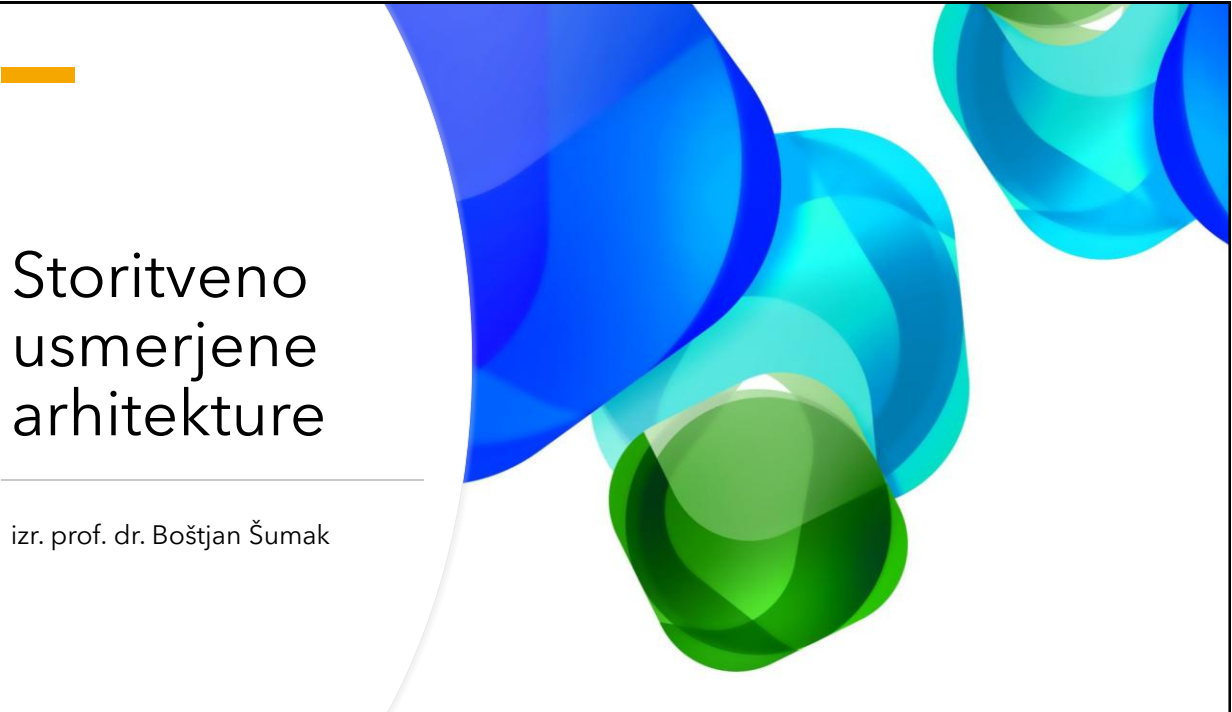




Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1



Razvoj storitev na osnovi ogrodja gRPC

2

Ogrodje gRPC - izvor

- Ogrodje gRPC je nastalo kot **odprtokodna evolucija** interne RPC infrastrukture **Stubby**
 - V podjetju Google so ga uporabljali za povezovanje velikega števila storitev znotraj podatkovnih centrov.
- Podjetje Google je gRPC **javno predstavilo in odprtokodno objavilo leta 2015**,
 - z namenom zagotoviti **učinkovit RPC** za povezovanje mikrorstitev v **različnih programskih jezikih** in **izvajalnih okoljih**
- Od takrat se je gRPC razvijal kot odprtokodni projekt in se razširil tudi **izven organizacije Google**

3

Ogrodje gRPC - osnovne lastnosti

- **Odprtokodno, visoko zmogljivo RPC ogrodje**, primerno za porazdeljene sisteme; z vtičniki podpira **uravnoveženje obremenitve, sledenje, preverjanje zdravja in avtentikacijo**
- **Uradne knjižnice v 11 programskih jezikih** (npr. C/C++, C#, Java, Go, Kotlin, Node.js, Python, PHP, Ruby, Dart, Objective-C)
- Komunikacija temelji na **HTTP/2** in podpira štiri tipe klicev: **unary, strežniško pretakanje, odjemalsko pretakanje, dvosmerno pretakanje**
- Privzeto uporablja **Protocol Buffers** kot **IDL** in sledi pristopu **contract-first** (definicija storitev in sporočil v *.proto*, nato generiranje odjemalcev/strežnikov)

4

Ogrodje gRPC je primerno za

- Rešitve, kjer sta pomembni **nizka zakasnitev** in **zanesljiva komunikacija** (tudi s podporo za pretakanje podatkov).
- **Komunikacijo med storitvami/mikrostoritvami** (*service-to-service*), zlasti v okoljih z veliko klici in visokimi zahtevami po učinkovitosti.
- Povezovanje sistemov v **različnih programskih jezikih in na različnih platformah** (poliglotska okolja).
- Povezovanje **podatkovnih centrov** in notranjih sistemov z velikim številom storitev.
- Povezovanje **mobilnih odjemalcev** z zalednimi storitvami (učinkovit prenos in jasne pogodbe vmesnikov).
- Povezovanje **IoT/naprav na robu** z zalednimi sistemi, kjer so pomembni kompakten prenos, zanesljivost in skalabilnost.
- Povezovanje **spletnih odjemalcev** prek **gRPC-Web** (kadar želimo gRPC podoben pristop v brskalniku).

5

gRPC – osnovni principi

Storitve namesto objektov	Sporočila namesto referenc na objekte	Pokritost in preprostost	Brezplačno in odprto	Interoperabilnost
Zmogljivost	Večnivojska zasnova	Agnostična vsebina (payload)	Pretakanje podatkov (streaming)	Preklic in časovna omejitev
Nadzor toka	Sposobnost razširjanja na osnovi vtičnikov	Izmenjava metapodatkov	Standardizirane statusne kode	...

Povzeto po: <https://grpc.io/blog/principles/>

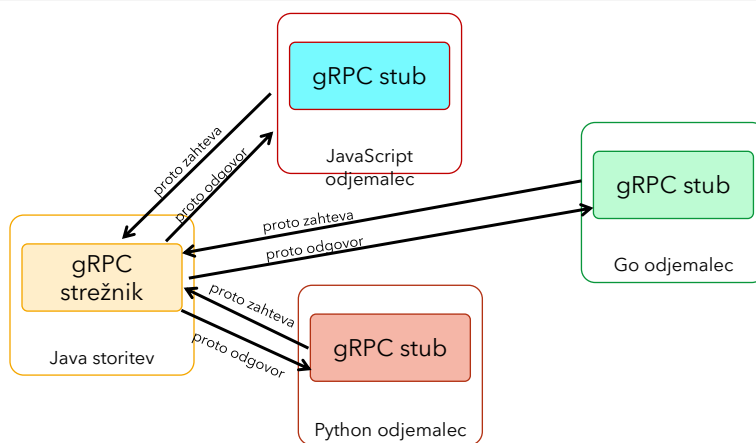
6

Ogrodje gRPC - interoperabilnost

- gRPC poleg zmogljivosti omogoča tudi visoko **interoperabilnost** med storitvami v različnih okoljih.
- Interoperabilnost temelji na **enotni definiciji vmesnika** v datotekah **.proto** (Protocol Buffers):
 - definicija storitev, metod in sporočil (sheme podatkov).
- Iz iste specifikacije se z orodji samodejno generira **domorodna (native) koda** za:
 - **strežnike** in **odjemalce** v različnih programskih jezikih,
 - kar zmanjša napake pri implementaciji in poenoti komunikacijo.
- Širša podpora v sodobnih jezikih in platformah ter dobra orodja za razvoj **spodbujajo uporabo gRPC** v porazdeljenih sistemih.

7

qRPC

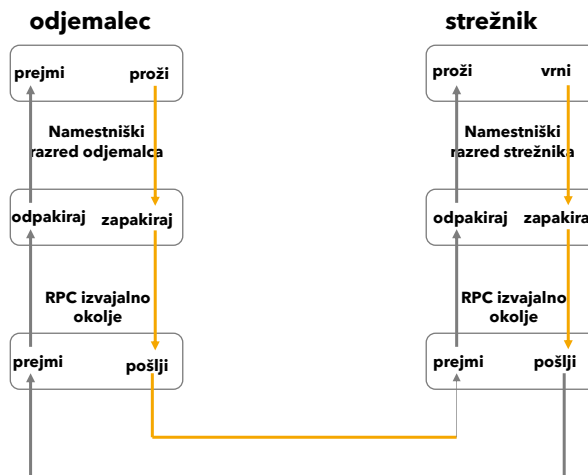


- Odjemalci gRPC lahko tečejo in se pogovarjajo v različnih okoljih in so spisani v različnih programskih jezikih
 - Tako lahko na primer enostavno ustvarimo strežnik gRPC v programskem jeziku Java in odjemalce s programskimi jeziki Go, Python, Ruby, JavaScript, Dart, Kotlin, itd.

8

gRPC - osnovni princip delovanja

- Na osnovi ogrodja gRPC lahko odjemalec proži metodo strežniške aplikacije, ki se nahaja na drugi napravi enako kot če bi bila aplikacija lokalno
- gRPC deluje po enakem principu kot že poznani sistemi tipa RPC



9

gRPC - osnovni princip delovanja

- Kot v drugih tehnologijah tipa RPC, protokol gRPC temelji na ideji **definicije vmesnika storitve s seznamom metod**
- Metoda storitve je določena z **vhodnimi sporočili** (zahteva) in **izhodnimi sporočili** (odgovor)
- Na strežniški strani, strežniška aplikacija implementira vmesnik in zagotavlja komunikacijo preko protokola gRPC
- Na odjemalski strani se zgradi namestniški objekt (stub, odjemalec), ki nudi enake metode kot strežnik

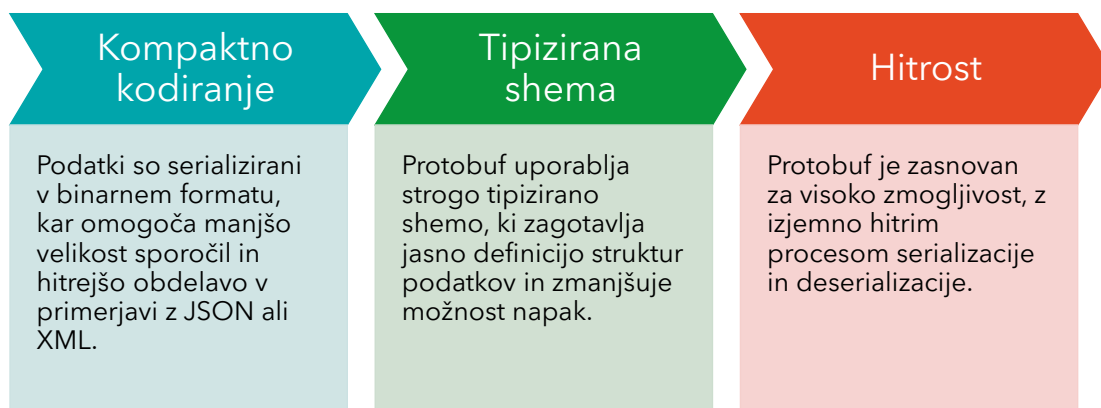
10

Protocol Buffers

- gRPC privzeto uporablja **Protocol Buffers** kot format sporočil in jezik za opis podatkov (**.proto**).
- **Odprtokodna tehnologija podjetja Google** za serializacijo in izmenjavo **strukturiranih podatkov** med različnimi sistemi.
- Podobno kot XML/JSON, vendar praviloma omogoča:
 - **manjša sporočila** (kompaktna binarna oblika),
 - **hitrejšo serializacijo/deserializacijo**,
 - **jasno shemo** in lažje generiranje kode.
- **Neodvisen od programskega jezika in platforme:** iz iste specifikacije lahko generiramo odjemalce in strežnike v različnih jezikih.

11

Ključne značilnosti Protocol Buffers



12

Protocol Buffers – osnovni koncepti

- Strukturo podatkov, ki jih želimo izmenjevati, definiramo v datoteki **.proto**.
- Podatki v Protocol Buffers so organizirani kot **sporočila (message)**.
- Sporočilo predstavlja logični zapis informacij, sestavljen iz **polj** (atributov) z določenim:
 - **imenom**,
 - **tipom**,
 - in **vrednostjo** (po potrebi tudi ponavljajočih se polj).

13

Protocol Buffers – primer definicije sporočila

```

syntax = "proto3";
package aips;
option java_package="si.feri.aips";
option java_outer_classname="StudentProto";

```

```

message Student{
  string ime = 1;
  string priimek = 2;
  string studijskiProgram = 3;
}

```

Polja imajo oznake (npr. 1, 2), ki služijo za identifikacijo med serializacijo.

14

Generiranje kode (Protocol Buffers)

- Iz datoteke **.proto** se lahko samodejno generira koda za različne jezike (npr. **Java, Python, C++, Go**).
- Uporabimo prevajalnik **protoc**, ki ustvari razrede/strukture za ciljni jezik, tipično z:
 - dostopnimi metodami za polja (npr. *getIme()*, *setIme()* ali *builder* pristop),
 - metodami za **serializacijo** in **deserializacijo** (pretvorba v/iz binarnega zapisa).

```
protoc --java_out=. student.proto
```

15

Podprti tipi podatkov (Protocol Buffers)

- **Osnovni tipi**
 - številčni: int32, int64, uint32, uint64, float, double
 - besedilni: string
 - logični: bool
 - binarni: bytes
- **Sestavljeni tipi**
 - enum – omejena množica vrednosti
 - **gnezdena sporočila** (*nested messages*)
 - **ponavljajoča se polja** (*repeated*) – sezname vrednosti

16

Preslikava skalarnih tipov .proto v tipe ciljnega programskega jezika

.proto tip	C++ tip	Java/Kotlin tip	Python tip	Go tip	Ruby tip	C# tip	PHP tip
double	double	Double	float	float64	Float	double	float
float	float	float	float	float32	Float	float	float
int32	int32	int	int	int32	Fixnum ali Bignum (kot zahtevano)	int	integer
int64	int64	long	int/long	int64	Bignum	long	integer/string
uint32	uint32	Int	int/long	uint32	Fixnum ali Bignum (kot zahtevano)	uint	Integer
uint64	uint64	Long	int/long	uint64	Bignum	ulong	integer/string
sint32	int32	Int	Int	int32	Fixnum ali Bignum (kot zahtevano)	int	Integer
sint64	int64	Long	int/long	int64	Bignum	long	integer/string
fixed32	uint32	Int	int/long	uint32	Fixnum ali Bignum (kot zahtevano)	uint	Integer
fixed64	uint64	Long	int/long	uint64	Bignum	ulong	integer/string
sfixed32	int32	Int	Int	int32	Fixnum ali Bignum (kot zahtevano)	int	Integer
sfixed64	int64	Long	int/long	int64	Bignum	long	integer/string
bool	bool	Boolean	Bool	bool	TrueClass/FalseClass	bool	Boolean
string	string	String	str/unicode	string	String (UTF-8)	string	String
bytes	string	ByteString	str (Python 2) bytes (Python 3)	[]byte	String (ASCII-8BIT)	ByteString	string

Prirejeno po: <https://developers.google.com/protocol-buffers/docs/proto3#scalar>

17

Protobuf – definicija gRPC storitve

- V gRPC je **storitev** definirana kot množica **RPC operacij (metod)**.
- Vsaka operacija je opisana s parom:
 - **vhodno sporočilo** (*request*),
 - **izhodno sporočilo** (*response*),
 ki sta definirana v **.proto** datoteki.
- Odjemalec mora poznati strukturo zahteve in odgovora – to zagotovi **pogodba** v .proto, iz katere se nato generira koda za odjemalca in strežnik.

18

Protobuf – struktura definicije gRPC storitve v .proto

- **Sintaksa:** definicija sledi pravilom **proto3**.
- **Ključna beseda *service*:** z njo definiramo ime storitve in seznam metod.
- **Metode storitve (*rpc*):** vsaka metoda je opisana z:
 - imenom metode,
 - **vhodnim sporočilom** (*request*),
 - **izhodnim sporočilom** (*response*).

19

Protocol Buffer – primer definicije gRPC storitve

```

syntax = "proto3";
package aips;
option java_package="si.feri.aips";
option java_outer_classname="StudentProto";

service Referat {
  rpc VpisStudenta (NovStudent) returns (VpisanStudent) {}
}

message NovStudent {
  string ime = 1;
  string priimek = 2;
  string studijskiProgram = 3;
}

message VpisanStudent {
  string vpisnaStevilka = 1;
  string ime = 2;
  string priimek = 3;
  string email = 4;
  string studijskiProgram = 5;
}

```

20

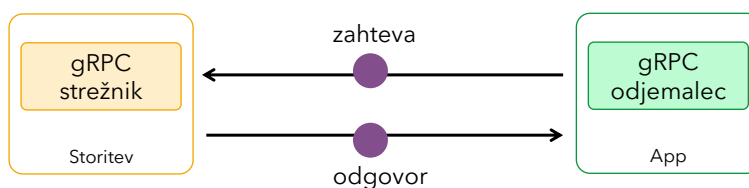
gRPC - modeli komunikacije

- gRPC podpira **4 modele komunikacije**:
 - **Unary (zahteva/odgovor)**: en zahtevek → en odgovor.
 - **Pretakanje s strani odjemalca (client streaming)**: odjemalec pošlje **tok sporočil**, strežnik vrne en odgovor.
 - **Pretakanje s strani strežnika (server streaming)**: odjemalec pošlje en zahtevek, strežnik vrne **tok sporočil**.
 - **Dvosmerno pretakanje (bidirectional streaming)**: odjemalec in strežnik hkrati izmenjujeta **tok sporočil v obe smeri**.

Novi načini komunikacije na osnovi podatkovnih tokov

21

Komunikacija zahteva/odgovor (Unary)



- Najosnovnejši način komunikacije, ki je pogosto uporabljen tudi pri drugih sistemih oz. rešitvah tipa RPC.
- 1. Odjemalec preko HTTP pošlje sporočilo tipa zahteva na strežnik,
- 2. Strežnik po procesiranju zahteve odgovori s sporočilom tipa odgovor
 - s tem se enosmerna komunikacija zaključí.
- Tipični primer sinhrona komunikacije, ki jih poznamo pri različnih oblikah spletno naravnanih aplikacijskih programskih vmesnikih.

22

Primer: zahteva/odgovor (Unary)

```

syntax = "proto3";
package kis;

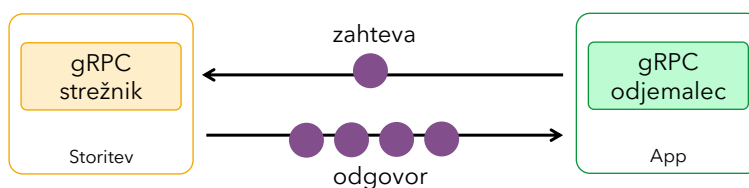
service Obvestila {
  rpc Obvesti (ObvestiloZahteva) returns (ObvestiloOdgovor) {}
}

message ObvestiloZahteva{
  string id = 1;
  string vsebina = 2;
}
message ObvestiloOdgovor{
  string id = 1;
  string corid = 2;
  string vsebina = 3;
}

```

23

Pretakanje s strani strežnika



1. Odjemalec pošlje sporočilo tipa zahteva
 2. Strežnik vzpostavi komunikacijski kanal, preko katerega pošlje odjemalcu zaporedje sporočil tipa odgovor
 - Odjemalec ohrani aktivno povezavo dokler ne prejme sporočilo/signal, da je operacija zaključena
- Ogrodje gRPC skrbi za pravilno in zaporedno razvrščanje poslanih sporočil znotraj aktivne povezave

24

Primer: Pretakanje s strani strežnika

```

syntax = "proto3";
package kis;

service Obvestila {
  rpc Obvesti (ObvestiloZahteva) returns (stream ObvestiloOdgovor) {}
}

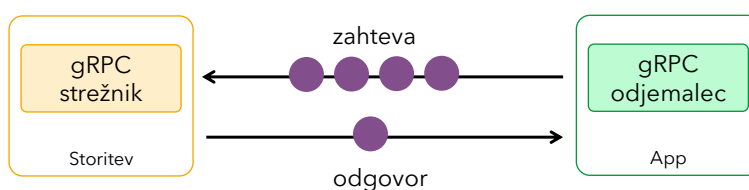
message ObvestiloZahteva {
  string id = 1;
  string vsebina = 2;
}

message ObvestiloOdgovor {
  string id = 1;
  string corid = 2;
  string vsebina = 3;
}

```

25

Pretakanje s strani odjemalca



1. Odjemalec odpre povezavo oz. kanal s strežnikom, preko katerega pošlje tok sporočil tipa zahteva
 - Ko odjemalec zaključi s pošiljanjem, čaka odgovor iz strani strežnika, da je prejel sporočila in pošlje signal o zaključitvi toka podatkov
2. Komunikacija se zaključi z odgovorom - končnim sporočilom tipa odgovor iz strani strežnika

26

Primer: Pretakanje s strani odjemalca

```

syntax = "proto3";
package kis;

service Obvestila {
  rpc Obvesti (stream ObvestiloZahteva) returns (ObvestiloOdgovor) {}
}

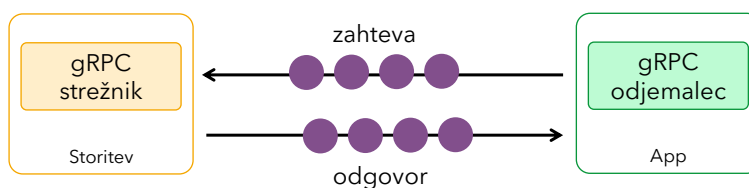
message ObvestiloZahteva {
  string id = 1;
  string vsebina = 2;
}

message ObvestiloOdgovor {
  string id = 1;
  string corid = 2;
  string vsebina = 3;
}

```

27

Dvosmerno pretakanje podatkov



- Zadnji možen način komunikacije na osnovi pretoka podatkov je dvosmerni tok podatkov
 - Podatkovna tokova delujeta neodvisno eden od drugega, kar omogoča neodvisno in sočasno pošiljanje sporočil iz obeh strani
 - Odvisno od implementacije lahko strežnik sicer počaka na zadnje sporočilo iz strani odjemalca in nato pošlje sporočila odjemalcu ali pa sproti in izmenično pošilja sporočila do odjemalca
- Vrstni red se v obeh podatkovnih tokovih ohrani v takšnem zaporedju, kot so bila sporočila oddana na komunikacijski kanal

28

Primer: Dvosmerno pretakanje podatkov

```

syntax = "proto3";
package kis;

service Obvestila {
  rpc Obvesti (stream ObvestiloZahteva) returns (stream ObvestiloOdgovor) {}
}

message ObvestiloZahteva {
  string id = 1;
  string vsebina = 2;
}
message ObvestiloOdgovor {
  string id = 1;
  string corid = 2;
  string vsebina = 3;
}

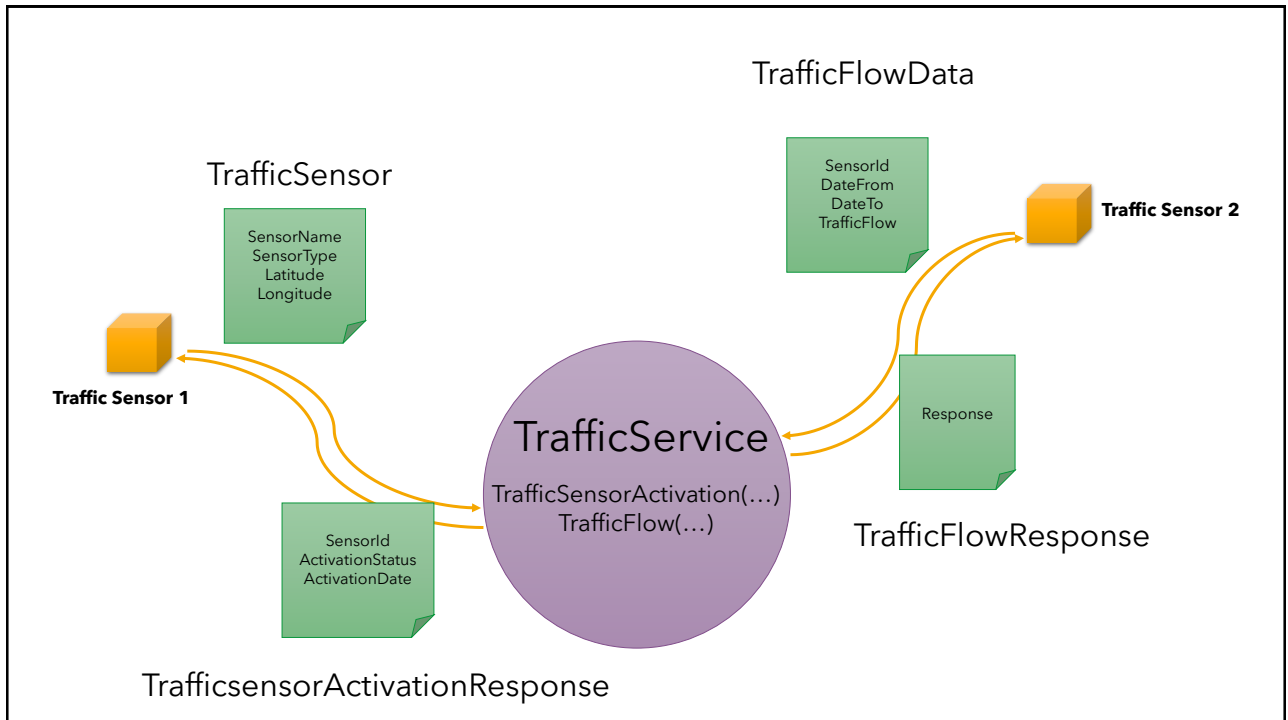
```

29

Primer rešitve

- Primer preproste rešitve na osnovi programskega jezika Java
- Izgradnja strežnika
- Preizkus storitve

30



31

Ustvarimo nov projekt

```
mvn -B archetype:generate \
  -DgroupId=si.feri.soa \
  -DartifactId=traffic-grpc \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.4
```

32


```

syntax="proto3";
package trafficService;
option java_multiple_files = true;

//storitev za registracijo senzorjev in poročanje o prometu
service TrafficService {
  //registracija novega senzorja
  rpc TrafficSensorActivation (TrafficSensor) returns
    (TrafficSensorActivationResponse);
  //poročanje prometa
  rpc TrafficFlow (TrafficFlowData) returns (TraficFlowResponse);
}

message TrafficSensor {
  string sensor_name = 1;
  string sensor_type = 2;
  double latitude = 3;
  double longitude = 4;
}

message TrafficSensorActivationResponse {
  string sensor_id = 1;
  string activation_status = 2;
  string activation_date = 3;
}

message TrafficFlowData {
  string sensor_id = 1;
  string datum_od = 2;
  string datum_do = 3;
  double flow = 4;
}

message TraficFlowResponse {
  string response = 1;
}

```

V mapo
main/proto
dodamo
trafficService.proto

33

Knjižnice

```

<properties>
  <maven.compiler.release>17</maven.compiler.release>
  <grpc.version>1.78.0</grpc.version>
  <protobuf.version>3.25.8</protobuf.version>
</properties>

```

```

<dependencyManagement>
<dependencies>
<dependency>
  <groupId>io.grpc</groupId>
  <artifactId>grpc-bom</artifactId>
  <version>${grpc.version}</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

```

```

<dependencies>
  <!-- gRPC runtime -->
  <dependency>
    <groupId>io.grpc</groupId>
    <artifactId>grpc-netty-shaded</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.grpc</groupId>
    <artifactId>grpc-protobuf</artifactId>
  </dependency>
  <dependency>
    <groupId>io.grpc</groupId>
    <artifactId>grpc-stub</artifactId>
  </dependency>
  <!-- Protobuf runtime -->
  <dependency>
    <groupId>com.google.protobuf</groupId>
    <artifactId>protobuf-java</artifactId>
    <version>${protobuf.version}</version>
  </dependency>

```

```

  <!-- pogosto potrebno zaradi @Generated v generirani kodi -->
  <dependency>
    <groupId>javax.annotation</groupId>
    <artifactId>javax.annotation-api</artifactId>
    <version>1.3.2</version>
  </dependency>
</dependencies>

```

34

```

<build>
<extensions>
<extension>
<groupId>kr.motd.maven</groupId>
<artifactId>os-maven-plugin</artifactId>
<version>1.7.1</version>
</extension>
</extensions>
<plugins>
<plugin>
<groupId>org.xolstice.maven.plugins</groupId>
<artifactId>protobuf-maven-plugin</artifactId>
<version>0.6.1</version>
<configuration>
<protocArtifact>
com.google.protobuf:protoc:${protobuf.version}:exe:${os.detected.classifier}
</protocArtifact>
<pluginId>grpc-java</pluginId>
<pluginArtifact>
io.grpc:protoc-gen-grpc-java:${grpc.version}:exe:${os.detected.classifier}
</pluginArtifact>
</configuration>
<executions>
<execution>
<goals>
<goal>compile</goal>
<goal>compile-custom</goal>
</goals>
</execution>
</executions>
</plugin>
<plugin>
<groupId>org.codehaus.mojo</groupId>
<artifactId>exec-maven-plugin</artifactId>
<version>3.3.0</version>
</plugin>
</plugins>
</build>

```

Konfiguracija plugin-a za generiranje kode

mvn -q -DskipTests compile

35

Generirana koda

```

v generated-sources
v protobuf
v grpc-java / trafficService
J TrafficServiceGrpc.java
v java / trafficService
J TrafficFlowData.java
J TrafficFlowDataOrBuilder.java
J TrafficSensor.java
J TrafficSensorActivationResponse.java
J TrafficSensorActivationResponseOrBuilder.java
J TrafficSensorOrBuilder.java
J TrafficService.java

```

36

Implementacija storitve

- V paket **si.feri.soa.service** dodamo razred `TrafficSensorServiceImpl`:

```
package si.feri.soa.service;

import java.time.OffsetDateTime;
import java.util.UUID;
import io.grpc.stub.StreamObserver;
import trafficservice.TrafficFlowData;
import trafficservice.TrafficFlowResponse;
import trafficservice.TrafficSensorActivationResponse;
import trafficservice.TrafficSensorData;
import trafficservice.TrafficServiceGrpc.TrafficServiceImplBase;

public class TrafficSensorServiceImpl extends TrafficServiceImplBase {

    @Override
    public void trafficSensorActivation(TrafficSensorData request,
        StreamObserver<TrafficSensorActivationResponse> responseObserver)
    {
        //implementacija metode
    }

    @Override
    public void trafficFlow(TrafficFlowData request, StreamObserver<TrafficFlowResponse> responseObserver)
    {
        //implementacija metode
    }
}
```

37

Implementacija storitve

trafficSensorActivation

```
@Override
public void trafficSensorActivation(TrafficSensorData request,
    StreamObserver<TrafficSensorActivationResponse> responseObserver) {

    String sensorId = UUID.randomUUID().toString();

    TrafficSensorActivationResponse response =
        TrafficSensorActivationResponse.newBuilder()
            .setSensorId(sensorId)
            .setActivationStatus("OK")
            .setActivationDate(OffsetDateTime.now().toString())
            .build();

    responseObserver.onNext(response);
    responseObserver.onCompleted();
}
```

38

Implementacija storitve

trafficFlow

```
@Override
public void trafficFlow(TrafficFlowData request,
    StreamObserver<TrafficFlowResponse> responseObserver) {

    String odgovor = "Prejel informacije o prometu" + request.getFlow()
        + " za senzor=" + request.getSensorId()
        + " [" + request.getDatumOd() + " - " + request.getDatumDo() + "]);

    TrafficFlowResponse response = TrafficFlowResponse.newBuilder()
        .setResponse(odgovor)
        .build();

    responseObserver.onNext(response);
    responseObserver.onCompleted();

}
```

39

Implementacija strežnika

- V paket **si.feri.soa.server** dodamo razred TrafficSensorServer:

```
package si.feri.soa.server;

import java.io.IOException;
import io.grpc.Server;
import io.grpc.ServerBuilder;
import si.feri.soa.service.TrafficSensorServiceImpl;

public class TrafficSensorServer {

    public static void main(String[] args) throws IOException, InterruptedException {

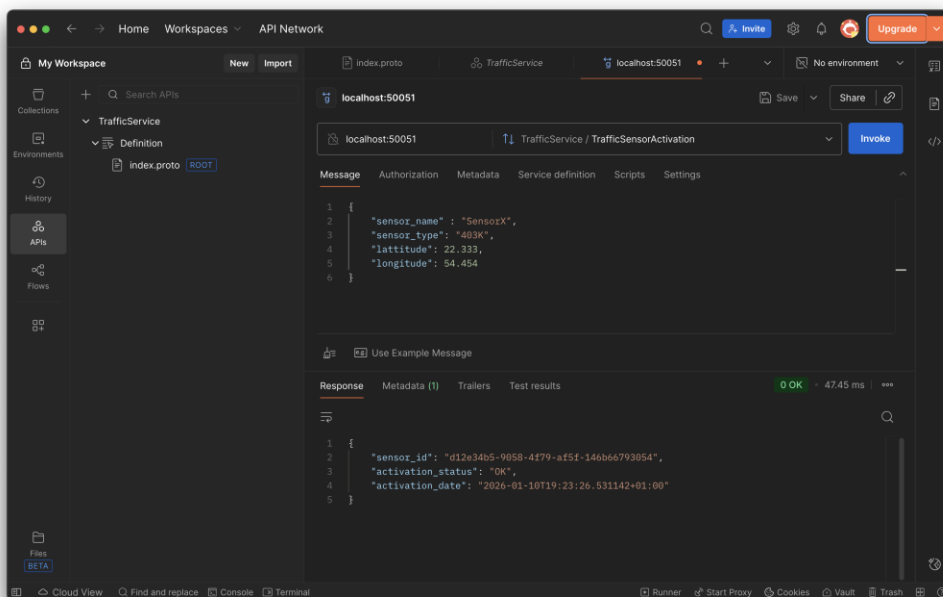
        int port = 50051;
        Server server = ServerBuilder.forPort(port).addService(new TrafficSensorServiceImpl())
            .build().start();
        System.out.println("gRPC strežnik teče na portu " + port);

        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            System.out.println("Zaustavljam gRPC strežnik...");
            server.shutdown();
        }));

        server.awaitTermination();
    }
}
```

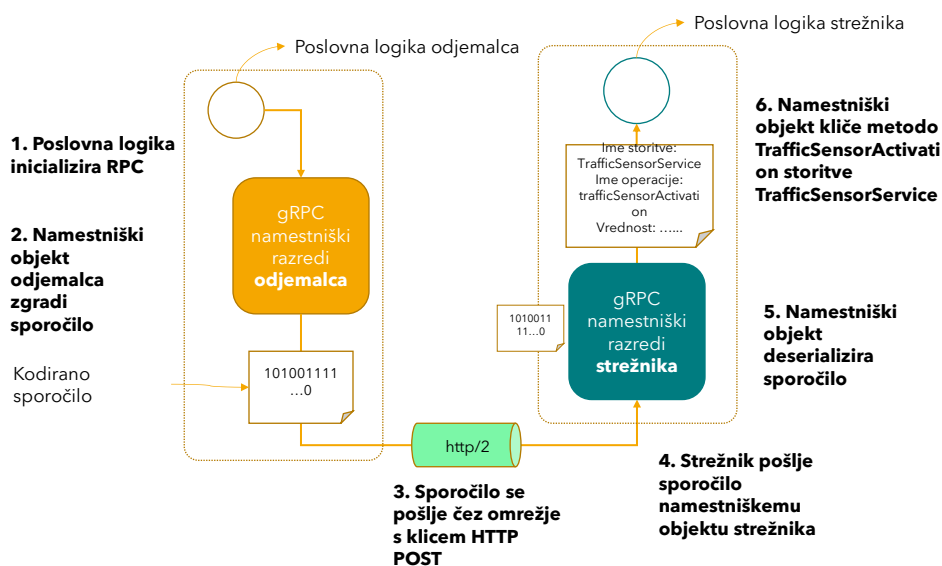
40

Testiranje storitve v orodju Postman



41

Proženje metode storitve na osnovi gRPC



Odgovor se izvede podobno v obratni smeri nazaj.

42

Sinhroni odjemalec

- V tipičnem klicu gRPC se izvedejo naslednji koraki:
 - Odjemalec začne delovni tok z inializacijo klica RPC
 - Ko se klic RPC začne, namestniški objekt odjemalca zakodira sporočilo v binarni format, ustvari zahtevo HTTP POST zahtevo in v telo zahteve vključi sporočilo.
 - Sporočilo se pošlje preko kanala. Kanal gRPC zagotavlja povezavo s strežnikom gRPC na definiranem naslovu in portu.
 - Na strežniški strani strežnik preda kodirano sporočilo avtomatsko generiranemu namestniškemu objektu strežniške aplikacije
 - Po prejemu sporočila, namestniški objekt strežnika sporočilo deserializira v podatkovno strukturo, specifično za ciljni programski jezik
 - V zadnjem koraku namestniški objekt strežnika izvede klic metode storitve in prenese razpoznano sporočilo
- Podobno se zgodi pri vračanju odgovora iz strani strežnika, kjer se sporočilo odgovor zakodira v binarno obliko in pošlje na stran odjemalca, kjer sporočilo deserializaria namestniški razred na strani odjemalca

43

```
package si.feri.soa.client;

import io.grpc.ManagedChannel;
...
public class TrafficSensorClient {

    public static void main(String[] args) {

        ManagedChannel channel = ManagedChannelBuilder
            .forAddress("localhost", 50051).usePlaintext().build();
        try {
            TrafficServiceGrpc.TrafficServiceBlockingStub stub =
                TrafficServiceGrpc.newBlockingStub(channel);
            // 1) Aktivacija senzorja
            TrafficSensorData sensor = TrafficSensorData.newBuilder()
                .setSensorName("Sensor-1")
                .setSensorType("INDUKCIJSKA_ZANKA")
                .setLatitude(46.5547)
                .setLongitude(15.6459).build();

            TrafficSensorActivationResponse activation = stub.trafficSensorActivation(sensor);
            System.out.println("Activation response: " + activation);

            // 2) TrafficFlow
            TrafficFlowData flowData = TrafficFlowData.newBuilder()
                .setSensorId(activation.getSensorId())
                .setDatumOd("2026-01-10T00:00:00+01:00")
                .setDatumDo("2026-01-10T23:59:59+01:00")
                .setFlow(123).build();
            TrafficFlowResponse flowResp = stub.trafficFlow(flowData);
            System.out.println("TrafficFlow response: " + flowResp.getResponse());
        } finally {
            channel.shutdownNow();
        }
    }
}
```

Primer
sinhronega
odjemalca gRPC
(TrafficSensorClient)

Za sinroni klic ustvarimo
namestniški object oz.
stub z metodo
newBlockingStub razreda
TrafficServiceGrpc

44

Asinhrono proženje gRPC

- V večini primerov je sicer sinhroni klic operacije ok, a sinhroni klic RPC blokira glavno nit aplikacije dokler ne dobi odgovor iz strani strežnika
- Ogrodje gRPC zagotavlja tudi vmesnike za asinhrono proženje zahteve – ki jih je potrebno implementirati in dodati ob proženju zahteve
- Callback handler se pokliče v ločeni niti, ko namestniški objekt dobi odgovor iz strani strežnika
- Med čakanjem odgovora glavna nit ni blokirana in se aplikacija normalno izvaja naprej
- Da implementiramo asinhroni klic, uporabimo razredno metodo **newStub** razreda **ImeStoritveGrpc**

45

Asinhrono proženje gRPC

- Asinhrono proženje lahko izvedemo na 2 načina:
 - Asinhroni Client Stub
 - Implementiramo callback handler na osnovi implementacije vmesnika **StreamObserver<T>**
 - Asinhroni Future Client Stub
 - Implementiramo vmesnik **FutureCallback<T>**

46

```
package si.feri.soa;

import io.grpc.stub.StreamObserver;

public class TrafficSensorActivationCallback implements
StreamObserver<TrafficSensorActivationResponse> {
```

TrafficSensorActivationCallback

```
    @Override
    public void onCompleted() {
        System.out.println("Komunikacija zaključena");
    }

    @Override
    public void onNext(TrafficSensorActivationResponse trafficSensorActivationResponse) {
        System.out.println("Senzor aktiviran: id=" +
            trafficSensorActivationResponse.getSensorId() +
            ", datum: " + trafficSensorActivationResponse.getActivationDate() +
            ", status = " + trafficSensorActivationResponse.getActivatonStatus());
    }

    @Override
    public void onError(Throwable throwable) {
        System.out.println("Napaka: " + throwable.getMessage());
    }
}
```

47

```
package si.feri.soa;
import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;
import java.util.concurrent.TimeUnit;
```

Primer sinhronega odjemalca gRPC (TrafficSensorAsyncClient)

```
public class TrafficSensorAsyncClient {
    public static void main(String[] args) throws InterruptedException {
        ManagedChannel kanal = ManagedChannelBuilder
            .forAddress("localhost", 9090)
            .usePlaintext()
            .build();

        TrafficSensorServiceGrpc.TrafficSensorServiceStub client =
            TrafficSensorServiceGrpc.newStub(kanal);

        TrafficSensor data = TrafficSensor.newBuilder()
            .setSensorName("SensorY")
            .setSensorType("Type X")
            .setLongitude("22.333")
            .setLatitude("43.211").build();

        client.trafficSensorActivation(data, new TrafficSensorActivationCallback());
        kanal.awaitTermination(10, TimeUnit.SECONDS);
    }
}
```

Za asinroni klic ustvarimo
namestniški object oz. stub z
metodo **newStub** razreda
TrafficSensorServiceGrpc

Ob asinhronem klicu dodamo
primerek razreda, ki
implementira vmesnik
StreamObserver<T>

48


```
package si.feri.soa;
```

VpisanStudentFutureCallback

```
import com.google.common.util.concurrent.FutureCallback;
```

```
public class TrafficSensorFutureCallback implements
```

```
FutureCallback<TrafficSensorActivationResponse> {
```

```
    @Override
```

```
    public void onSuccess(TrafficSensorActivationResponse  
trafficSensorActivationResponse) {
```

```
        System.out.println("Senzor aktiviran");
```

```
        System.out.println("ID = " + trafficSensorActivationResponse.getSensorId());
```

```
        System.out.println("Datum aktivacije = " +
```

```
trafficSensorActivationResponse.getActivationDate());
```

```
        System.out.println("Status = " +
```

```
trafficSensorActivationResponse.getActivatonStatus());
```

```
    }
```

```
    @Override
```

```
    public void onFailure(Throwable throwable) {
```

```
        System.out.println("Napaka: " + throwable.getMessage());
```

```
    }
```

```
}
```

49

```
package si.feri.soa;
```

```
import com.google.common.util.concurrent.Futures;
```

```
import com.google.common.util.concurrent.ListenableFuture;
```

```
import io.grpc.ManagedChannel;
```

```
import io.grpc.ManagedChannelBuilder;
```

```
import java.util.concurrent.ExecutorService;
```

```
import java.util.concurrent.Executors;
```

```
import java.util.concurrent.TimeUnit;
```

```
public class TrafficSensorFutureAsyncClient {
```

```
    public static void main(String[] args) throws InterruptedException {
```

```
        ExecutorService executorService = Executors.newFixedThreadPool(10);
```

```
        ManagedChannel kanal = ManagedChannelBuilder
```

```
            .forAddress("localhost", 9090)
```

```
            .usePlaintext()
```

```
            .build();
```

```
        TrafficSensor sensor = TrafficSensor.newBuilder()
```

```
            .setSensorName("SensorX").setSensorType("TypeX")
```

```
            .setLongitude("34.3222").setLongitude("39.43322").build();
```

```
        TrafficSensorServiceGrpc.TrafficSensorServiceFutureStub client
```

```
            = TrafficSensorServiceGrpc.newFutureStub(kanal);
```

```
        ListenableFuture<TrafficSensorActivationResponse> odg
```

```
            = client.trafficSensorActivation(sensor);
```

```
        Futures.addCallback(odg, new TrafficSensorFutureCallback(), executorService);
```

```
        kanal.awaitTermination(10, TimeUnit.SECONDS);
```

```
        kanal.shutdown();
```

```
    }
```

asinhroni odjemalec gRPC
(ReferatFutureAsyncClient)

Za asinroni klic ustvarimo namestniški
object oz. stub z metodo `newFutureStub`
razreda `TrafficSensorServiceGrpc`

Ob asinhronem klicu dodamo primerek razreda,
ki implementira vmesnik
`FutureCallback<VpisanStudent>`

50